

AUDIT TECHNIQUE COMPLET

ONIT-PNG

2026

Audit complet de la plateforme ONIT-PNG couvrant le backend, le frontend et la preparation au deploiement en production. Analyse des vulnerabilites, performances, securite et recommandations detaillees.

ARPT - Republique de Guinee

Mai 2026

Table des matieres

1. Resume executif	3
2. Audit backend	4
2.1 Architecture et stack technique	4
2.2 Routes API et evaluation	4
2.3 Authentification et autorisation	5
2.4 Base de donnees et performances	6
2.4.1 Absence totale d'index	6
2.4.2 Requetes critiques de performance	6
2.5 Securite	7
2.5.1 Vulnerabilites critiques	7
2.6 Gestion des erreurs et observabilite	8
3. Audit frontend	8
3.1 Architecture et composants	8
3.2 Securite de type TypeScript	9
3.3 Accessibilite (a11y)	9
3.4 Performance frontend	10
3.5 Gestion d'etat et formulaires	10
3.6 Responsive design	10
4. Preparation a la production	11
4.1 Infrastructure Docker	11
4.2 CI/CD et tests	11
4.3 Monitoring et observabilite	11
4.4 Fonctionnalites manquantes pour la production	12
5. Recommandations detaillees	13
5.1 Priorite P0 - Correctifs critiques (avant production)	13

P0.1 - Remplacer les aggregations en memoire par des requetes base de donnees	. 13
P0.2 - Ajouter les index de base de donnees	 13
P0.3 - Implementer le rate limiting sur tous les endpoints	 14
P0.4 - Ajouter les en-tetes de securite manquants	 14
P0.5 - Ajouter les Error Boundaries React	 14
5.2 Priorite P1 - Correctifs importants (semaine 1-2)	 14
P1.1 - Verrouillage de compte et protection timing	 14
P1.2 - Appliquer le RLS sur les operations d'ecriture	 15
P1.3 - Implementer le logging structure	 15
P1.4 - Configurer la pipeline CI/CD	 15
P1.5 - Corriger la securite de type TypeScript	 15
5.3 Priorite P2 - Ameliorations a moyen terme (mois 1-2)	 15
P2.1 - Migration vers PostgreSQL	 15
P2.2 - Implementer React Query et le code-splitting	 16
P2.3 - Implementer les formulaires avec react-hook-form + Zod	 16
P2.4 - Ajouter l'accessibilite (a11y)	 16
P2.5 - Ajouter le cache de reponses API	 16
5.4 Priorite P3 - Ameliorations a long terme (mois 3+)	 16
6. Plan d'action par phases	17

1. Resume executif

L'audit complet de la plateforme ONIT-PNG (Observatoire National Intelligent des Telecommunications - Plateforme Nationale de Guinee) a ete realise afin d'evaluer la qualite du backend, du frontend et la preparation au deploiement en production. Cette plateforme, developpee pour l'ARPT (Autorite de Regulation des Postes et des Telecommunications) de la Republique de Guinee, assure la supervision, la notation et le reporting de la qualite de service (QoS) des trois operateurs de telecommunication (Orange, MTN, Celcom) sur les 8 regions administratives du pays.

L'audit a revele une architecture globalement coherente avec des choix technologiques modernes (Next.js 16, Prisma ORM, NextAuth v4, shadcn/ui), un systeme RBAC + RLS bien concu, et une couverture de validation d'entrees correcte via Zod. Cependant, des problemes critiques de performance, de securite et d'observabilite menacent la viabilite de la plateforme en production. Les trois problemes les plus urgents sont : les requetes de base de donnees en lecture complete de table (full table scan) dans les endpoints /api/dashboard, /api/map et /api/qos ; l'absence de rate limiting sur les endpoints publics et de modification ; et l'absence totale de tests automatises et de pipeline CI/CD.

Categorie	Note	Commentaire
Authentification	7/10	Implémentation solide, absence de verrouillage de compte et protection timing
Autorisation (RBAC)	7/10	Bon modèle, manque de vérification RLS sur les opérations d'écriture
Validation des entrées	7/10	Zod sur toutes les mutations, HTML stripping insuffisant, pas de validation GET
Gestion des erreurs	4/10	Incohérent, pas de monitoring, fuite d'erreurs dans certaines réponses
Performance	2/10	Full table scans, requêtes N+1, aucun index, pas de cache
En-têtes de sécurité	6/10	Bonnes bases, absence de CSP et HSTS
Rate Limiting	2/10	Seulement sur login, en mémoire, endpoints publics non protégés
Conception base de données	5/10	Bon schéma, aucun index, SQLite limite la scalabilité
Piste d'audit	6/10	Présente sur la plupart des opérations, IP/UA non capturées
Observabilité	1/10	Console.error uniquement, pas d'APM, pas de logging structuré
Frontend - Qualité	5/10	Bons dashboards, mais aucune Error Boundary, types ignorés, pas de cache
Frontend - Accessibilité	2/10	Aucun attribut ARIA, pas de navigation clavier, pas de support lecteur d'écran

Tests automatisés	1/10	Aucun test unitaire, intégration ou E2E, script bash manuel uniquement
-------------------	------	--

Tableau 1 : Synthèse des scores par catégorie

2. Audit backend

2.1 Architecture et stack technique

La plateforme repose sur Next.js 16 avec App Router, TypeScript 5, Prisma ORM 6 avec SQLite, NextAuth v4 (stratégie JWT, fournisseur Credentials), et une interface utilisateur basée sur TailwindCSS 4 + shadcn/ui. L'application est conçue comme une SPA (Single Page Application) avec un système d'onglets côté client, ce qui constitue un choix architectural discutable pour une plateforme institutionnelle qui bénéficierait d'un routage basé sur le système de fichiers pour le SEO et le partage de liens profonds.

Composant	Technologie	Version
Framework	Next.js (App Router)	16
Langage	TypeScript	5
ORM	Prisma	6
Base de données	SQLite	-
Authentification	NextAuth v4 (JWT)	4
UI	TailwindCSS + shadcn/ui	4
Cartographie	Leaflet + React-Leaflet	1.9/5
Graphiques	Recharts	2.15
Validation	Zod	4
Déploiement	Docker (multi-stage) + Caddy	-

Tableau 2 : Stack technique du backend

2.2 Routes API et évaluation

La plateforme expose 12 endpoints API couvrant les fonctionnalités de dashboard, QoS, cartographie SIG, scoring, campagnes, mesures, alertes, rapports, gestion des utilisateurs, rôles et journaux d'audit. La plupart des endpoints appliquent correctement les vérifications d'authentification et les politiques RLS. Cependant, trois endpoints critiques (dashboard, map, qos) présentent des problèmes de

performance severs lies au chargement complet de tables en memoire suivi de filtrages en JavaScript.

Route	Methodes	Auth	Evaluation
/api/dashboard	GET	Public OK	Critique - full table scan
/api/map	GET	Public OK	Critique - full table scan
/api/qos	GET	Requise	Critique - 4+ requetes + filtrage JS
/api/campaigns	GET/POST	Requise	Bon - RLS filtree
/api/mesures	GET/POST/ PUT	Requise	N+1 sur bulk import
/api/scores	GET/POST	Partiel	Bon - upsert
/api/alerts	GET/POST/ PATCH	Partiel	Alerte - POST public sans rate limit
/api/reports	GET/POST	Partiel	Moyen
/api/users	GET/POST/ PATCH	Admin	Bon
/api/roles	GET	Admin	Bon
/api/audit-logs	GET	Audit	Moyen - limit 100
/api/auth/[...nextauth]	GET/POST	N/A	Bon

Tableau 3 : Catalogue des endpoints API et evaluations

2.3 Authentification et autorisation

Le systeme d'authentification repose sur NextAuth v4 avec une strategie JWT (duree de vie de 8 heures), un fournisseur Credentials (email + mot de passe), et un hachage bcrypt avec un facteur de cout de 12. Le mecanisme est globalement solide mais presente plusieurs vulnerabilites qui doivent etre corrigees avant la mise en production.

Vulnerabilite - Attaque par timing sur le login : La fonction authorize() effectue un findUnique puis bcrypt.compare. Si l'utilisateur n'existe pas, la reponse est immediate (rapide). Si l'utilisateur existe, bcrypt prend environ 100ms. Cette difference de temps permet a un attaquant de determiner si un email existe dans le systeme, facilitant les attaques d'enumeration d'utilisateurs.

Vulnerabilite - Pas de verrouillage de compte : Le middleware limite les tentatives par IP (5 tentatives / 15 minutes), mais le handler d'authentification ne verifie pas les tentatives echouees par compte. Un attaquant rotant les IPs peut brute-forcer les mots de passe indefiniment.

Vulnerabilite - Permissions figees dans le JWT : Les permissions sont integrees dans le JWT au moment du login. Si un admin revoque les permissions d'un utilisateur, l'ancien JWT reste valide pendant jusqu'a 8 heures. Il n'y a aucun mecanisme de revocation immediate des tokens.

Le systeme RBAC est bien concu avec 9 roles distincts (SUPER_ADMIN, DG, DGA, DIRECTEUR_TECHNIQUE, INGENIEUR_RF, ANALYSTE_QOS, AUDITEUR, OPERATEUR_READONLY, PUBLIC), des permissions granulaires (resource + action), et des politiques d'accès aux données (RLS) par portée (all, own_org, own_region, public_only). Cependant, le code RLS presente un bug potentiel dans la resolution des operateurs accessibles : la methode `getAccessibleOperators()` avec le scope 'own_org' utilise un fragil contains: `organization.split(' ')[0]` qui pourrait correspondre a des operateurs non intends. De plus, les operations d'écriture (POST/PUT/PATCH) ne verifient pas que l'utilisateur peut écrire pour l'operateur/la region specifie(e), ce qui constitue une faille dans l'application du RLS.

2.4 Base de donnees et performances

La base de donnees utilise SQLite via Prisma ORM avec 13 modeles bien structures. Le schema inclut des relations appropriees (User vers Role, Campagne vers Operateur/Region, MesureQoS vers Campagne, ScoreOperateur vers Operateur), des contraintes d'unicite (User.email, Operateur.code, Region.code), et une contrainte composite unique sur ScoreOperateur (operateurId + periode). Cependant, le schema comporte des lacunes critiques en termes de performance.

2.4.1 Absence totale d'index

Le schema Prisma ne declare aucun `@@index`. SQLite utilise uniquement des index implicites pour les champs `@unique`. Toutes les requetes filtrant sur `operateurId`, `regionId`, `campagneId`, `typeMesure`, `createdAt` ou `timestamp` effectuent des scans complets de table. Avec des milliers de mesures QoS, chaque requete sur ces champs parcourt l'integralite de la table, ce qui degrade lineairement les performances avec le volume de donnees.

Index recommandes a ajouter au schema Prisma :

MesureQoS : `@@index([operateurId])`, `@@index([regionId])`, `@@index([campagneId])`,
`@@index([timestamp])`, `@@index([typeMesure])`, `@@index([operateurId, regionId])`
AuditLog : `@@index([userId])`, `@@index([createdAt])`, `@@index([resource])`
Alerte : `@@index([operateurId])`, `@@index([regionId])`, `@@index([createdAt])`
DataAccessPolicy : `@@unique([roleType, resource])`

2.4.2 Requetes critiques de performance

Trois endpoints chargent l'integralite de la table `MesureQoS` en memoire puis effectuent des filtrages et aggregations en JavaScript. Cette approche est catastrophique pour les performances et la memoire :

Endpoint	Probleme	Impact
/api/dashboard	findMany() sans limite + filtrage JS (.filter(), .reduce()) pour couverture, stats par region, stats par operateur	OOM avec 10k+ mesures, temps de reponse en secondes
/api/map	findMany() sans limite + filtrage JS par region pour calculs QoS/couverture	Meme probleme que dashboard, payload JSON massif
/api/qos	4+ requetes paralleles + findMany avec includes + filtrage JS pour benchmarks	4x le probleme, surcharge memoire et CPU
PUT /api/mesures	N+1 : pour chaque ligne importee, 1-2 requetes de resolution (operateur, region)	20 000+ requetes pour 10 000 lignes CSV

Tableau 4 : Problemes de performance critiques par endpoint

Solution recommandee : Remplacer les aggregations en memoire par des requetes Prisma groupBy() ou du SQL brut. Exemple : au lieu de charger toutes les mesures puis filtrer en JS, utiliser db.measureQoS.groupBy({ by: ['regionId'], _avg: { scoreQoE: true }, _count: { id: true }, where: { rssi: { gt: -100 } } }). Pour le bulk import, pre-charger les operateurs et regions en memoire une seule fois, puis resoudre les IDs par lookup dans un Map au lieu de requetes individuelles.

2.5 Securite

2.5.1 Vulnerabilites critiques

L'audit a identifie plusieurs vulnerabilites de securite qui doivent etre corrigees en priorite avant tout deploiement en production. Les en-tetes de securite du middleware sont un bon point de depart (X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy, Permissions-Policy), mais des lacunes importantes subsistent.

Severite	Vulnerabilite	Description
CRITIQUE	Pas de rate limiting API	Seul le login est limite (5/15min). Tous les autres endpoints sont ouverts au DoS et au brute-force
CRITIQUE	POST /api/alerts sans auth	Endpoint public pour SIGNALEMENT_PUBLIC sans rate limiting, CAPTCHA ou verification. Risque de flood
CRITIQUE	Rate limiter en memoire	Le Map en memoire se reinitialise au redemarrage, ne se partage pas entre instances, fuite memoire potentielle
ELEVE	Pas de protection CSRF	Les routes API POST/PATCH/PUT n'ont pas de token CSRF. Un attaquant peut soumettre des formulaires depuis le navigateur d'un utilisateur connecte

ELEVE	Pas de Content-Security-Policy	Absence de l'en-tete CSP, premiere defense contre les attaques XSS
ELEVE	Pas de HSTS	Absence de l'en-tete Strict-Transport-Security pour les deploiements HTTPS
MOYEN	HTML stripping insuffisant	stripHtml() utilise une regex simple qui ne detecte pas les event handlers sans <>, les entites encodees, les expressions CSS
MOYEN	IP non capturee dans audit	logAudit() accepte ipAddress et userAgent mais aucune route API ne transmet ces valeurs
MOYEN	Mot de passe seed en clair	Admin@2026! visible dans seed.ts et le script de test

Tableau 5 : Vulnerabilites de securite identifiees

2.6 Gestion des erreurs et observabilite

La gestion des erreurs est l'un des points les plus faibles de la plateforme. Tous les handlers API utilisent le meme motif try/catch avec console.error et une reponse generique 500. Il n'y a pas de logging structure, pas de codes d'erreur, pas d'IDs de correlation, et aucune integration APM (Sentry, DataDog, etc.). En production, les erreurs seront invisibles et impossibles a diagnostiquer.

Fuite d'informations dans /api/scores : La reponse d'erreur du POST /api/scores expose le message d'erreur Prisma brut : { error: (upsertError as Error).message }. Ce message peut reveler des details du schema de base de donnees a un attaquant. Toutes les erreurs doivent retourner un message generique en production.

Le parser CSV est naif et ne gere pas les champs entre guillemets contenant des virgules, les guillemets echappes, ou les differents encodages. L'import en masse ignore silencieusement les lignes invalides avec seulement un compteur, sans rapport detaille des echecs. Le format de reponse d'erreur est inconsistent : certains endpoints retournent { error: string }, d'autres retournent { error: string, details: object }, sans enveloppe d'erreur standardisee.

3. Audit frontend

3.1 Architecture et composants

Le frontend est construit comme une SPA dans le cadre de l'App Router de Next.js, avec un seul point d'entree (page.tsx) et un systeme d'onglets cote client via OnitLayout. L'application comporte 9 dashboards (DG, QoS, SIG, Scoring, Audit, Reports, Public, Cyber, Admin), chacun implemente comme un composant React distinct. Les composants shadcn/ui (47 composants Radix) sont tous installes, mais beaucoup sont inutilises, ce qui ajoute un ballonnement significatif au bundle.

L'architecture frontend presente un paradoxe : plusieurs bibliotheques sont installees mais jamais utilisees. Zustand est installe mais aucun store n'existe. React Query est installe mais tous les fetchs utilisent useEffect + useState. React Hook Form et Zod sont installes mais les formulaires utilisent des useState manuels. next-intl est installe mais tout le texte est en dur en francais. Ce gachis de dependances ajoute entre 200 et 400 Ko au bundle JavaScript final sans aucun benefice.

3.2 Securite de type TypeScript

La securite de type TypeScript est systematiquement contournee. L'audit a identifie 44 instances de casts 'as Record' a travers 16 fichiers, principalement pour acceder aux proprietes de session (role, organization, permissions). Le fichier next-auth.d.ts definit correctement les types etendus pour Session.user, mais ces types sont ignores dans le code des composants. Cette pratique annule completement les benefices du typage statique et pourrait masquer des bugs critiques lors de refactoring.

Exemple du probleme :

Code actuel : `const userRole = (session?.user as Record<string, unknown>)?.role as string;`

Code correct : `const userRole = session?.user?.role; // Type-safe grace a next-auth.d.ts`

Il n'y a pas d'interfaces partagees entre le backend et le frontend. Chaque dashboard redeclare independamment des interfaces identiques (MapRegionData, MapPointData, OperatorData) au lieu d'importer depuis un fichier de types commun. Les reponses API sont toutes consommees via `await res.json()` sans aucune validation de type ou assertion, ce qui signifie que tout changement cote backend pourrait silently casser le frontend sans erreur de compilation.

3.3 Accessibilite (a11y)

L'accessibilite est le point le plus faible du frontend. Les composants customises (sidebar, notifications, menu utilisateur, cartes de metriques, graphiques SVG) ne contiennent aucun attribut ARIA. La barre laterale n'a pas de `role="navigation"`, les boutons d'onglets n'ont pas de `role="tablist/tab"`, le dropdown de notifications n'a pas de piege de focus. Les graphiques SVG n'ont ni `aria-label` ni elements title. Les indicateurs de severite reposent uniquement sur la couleur (points rouge/ambre/bleu) sans alternative textuelle. Il n'y a pas de lien 'allez au contenu principal' ni de gestion du focus dans la modale de login.

Les composants shadcn/ui bases sur Radix integrent correctement les attributs ARIA, mais le projet ne les utilise pas pour les elements d'interface customises. Cette lacune rend la plateforme inutilisable avec un lecteur d'ecran et non conforme aux normes WCAG 2.1, ce qui pourrait poser des problemes juridiques pour une plateforme gouvernementale.

3.4 Performance frontend

La performance frontend souffre de plusieurs problemes architecturaux. Tous les composants dashboard sont importes de maniere statique dans onit-layout.tsx via une map dashboardComponents, ce qui signifie que les 9 dashboards sont dans le meme bundle JavaScript malgre qu'un seul soit affiche a la fois. Chaque changement d'onglet demont le composant precedent et re-charge toutes les donnees depuis zero sans aucun cache. Les composants React.memo sont absents, provoquant des re-rendus complets a chaque changement d'etat. Le composant carte Leaflet se re-initialise completement a chaque changement de prop via un attribut key qui force le remontage.

Probleme	Impact	Solution
Pas de code-splitting des dashboards	Bundle massif, tous les dashboards charges inutilement	React.lazy + Suspense pour chaque dashboard
Pas de cache de donnees	Re-fetch complet a chaque changement d'onglet	React Query avec staleTime et cacheTime
Pas de React.memo	Re-rendus complets inutiles a chaque changement d'etat	Memoiser les composants dashboard et MetricCard
Dependances inutilisees (25+)	Bundle gonfle de 200-400 Ko inutiles	Supprimer zustand, next-intl, @mdxeditor, @dnd-kit, etc.
Carte Leaflet re-initialisee	Flash blanc et delai a chaque changement de filtre	Utiliser updateLayer au lieu de key pour forcer le remontage

Tableau 6 : Problemes de performance frontend et solutions

3.5 Gestion d'etat et formulaires

La gestion d'etat est entierement basee sur des useState locaux. Il n'y a pas d'etat global partage entre les composants, ce qui signifie que les donnees de carte sont recuperees independamment par les dashboards DG, SIG et Public, entrainant des requetes API triplicees. Les formulaires utilisent tous le meme motif manuel : useState pour chaque champ, verification ad-hoc des champs requis, pas de validation de format (email, telephone), pas de retour visuel de force de mot de passe. Les bibliotheques react-hook-form, @hookform/resolvers et zod sont pourtant installees mais completement inutilisees.

3.6 Responsive design

Le design responsive est globalement adequat pour les ecrans mobiles et desktop, avec une barre laterale collapsible, des grilles adaptatives (grid-cols-1 sm:grid-cols-2 lg:grid-cols-4), et des elements masques sur mobile (recherche, breadcrumb, nom utilisateur). Cependant, plusieurs problemes subsistent : la carte Leaflet a une hauteur fixe de 400px qui ne s'adapte pas au viewport ; les graphiques

utilisent des largeurs en pixels fixes (`width=500`, `width=200`) ; le dropdown de notifications a une largeur fixe de `w-80` qui peut déborder sur petits écrans ; et de nombreuses tailles de police sont extrêmement petites (`text-[9px]`, `text-[10px]`) sur mobile. Le hook `useIsMobile` existe dans `src/hooks/use-mobile.ts` mais n'est jamais utilisé dans les composants.

4. Preparation a la production

4.1 Infrastructure Docker

L'infrastructure Docker est raisonnablement bien configurée avec un Dockerfile multi-stage (`deps`, `builder`, `runner`), un utilisateur non-root (`nextjs:1001`), un mode standalone pour une empreinte minimale, et un health check sur `/api/auth/session`. Le `docker-compose.yml` définit un volume persistant pour la base SQLite et exige `NEXTAUTH_SECRET` via la syntaxe `${:?}`. Le Caddyfile configure un reverse proxy avec transmission des en-têtes `X-Real-IP` et `X-Forwarded-For`.

Cependant, plusieurs améliorations sont nécessaires. La base SQLite est un single-writer : sous charge concurrente (imports en masse, utilisateurs multiples), les écritures se serialiseront et pourront timeout. Le health check Docker utilise `/api/auth/session` au lieu d'un endpoint dédié qui vérifierait réellement la connectivité base de données. Il n'y a pas de configuration pour le scaling horizontal (plusieurs conteneurs), et le rate limiter en mémoire ne se partage pas entre les instances. Le script de démarrage exécute `'prisma db push --skip-generate'` à chaque démarrage, ce qui est dangereux en production car cette commande peut altérer le schéma de manière inattendue.

4.2 CI/CD et tests

Il n'existe aucune pipeline CI/CD configurée (pas de `.github/workflows`, `.gitlab-ci.yml`, ou `Jenkinsfile`). Il n'y a aucun framework de test installé (Jest, Vitest, Playwright, Cypress), aucun fichier de test (`.test.*` ou `.spec.*`), et aucun test E2E. Le seul artefact de test est un script bash manuel (`tests/api/test_api_endpoints.sh`) qui vérifie les codes de statut HTTP des endpoints via curl, sans vérifier le contenu des réponses, les autorisations RBAC, ou le filtrage RLS.

Risque critique : Sans tests automatisés, toute modification du code risque d'introduire des régressions invisibles. Le système RBAC/RLS, qui est le pilier de la sécurité de la plateforme, n'est testé par aucun test d'intégration. Un changement dans `rbac.ts` pourrait silencieusement octroyer des accès non autorisés sans qu'aucun test ne le détecte.

4.3 Monitoring et observabilité

L'observabilite est quasi inexistante. Les erreurs sont enregistrees uniquement via `console.error` sans format structure, sans code d'erreur, sans ID de correlation, et sans contexte de requete (chemin, utilisateur, parametres). Il n'y a aucune integration APM (Sentry, DataDog, New Relic), pas de metriques de performance (temps de reponse, taux d'erreur, utilisation memoire), pas de tableau de bord de monitoring, et pas de systeme d>alerting. En production, il sera impossible de detecter, diagnostiquer et resoudre les incidents dans des delais raisonnables.

4.4 Fonctionnalites manquantes pour la production

Categorie	Fonctionnalite manquante	Priorite
Securite	Content-Security-Policy header	Critique
Securite	Protection CSRF sur les routes API	Critique
Securite	Rate limiting sur tous les endpoints	Critique
Securite	Verrouillage de compte apres echecs repetes	Elevee
Securite	Endpoint de changement de mot de passe	Elevee
Securite	2FA/MFA pour les comptes admin	Moyenne
Architecture	Versionnage API (/api/v1/)	Elevee
Architecture	Health check dedie (DB + deps)	Elevee
Architecture	IDs de correlation / request ID	Elevee
Architecture	Documentation OpenAPI/Swagger	Moyenne
Donnees	Soft deletes au lieu de hard deletes	Moyenne
Donnees	Politique de retention des donnees	Moyenne
Donnees	Endpoint d'export/backup	Moyenne
Frontend	Error Boundaries React	Critique
Frontend	Pages d'erreur (error.tsx, not-found.tsx)	Elevee
Frontend	Internationalisation via next-intl	Basse
DevOps	Pipeline CI/CD	Critique
DevOps	Tests unitaires, integration, E2E	Critique
DevOps	Integration APM (Sentry/DataDog)	Elevee
DevOps	Logging structure (Pino/Winston)	Elevee

Tableau 7 : Fonctionnalites manquantes pour la production

5. Recommandations detaillees

Les recommandations suivantes sont classees par ordre de priorite, depuis les correctifs urgents qui doivent etre appliques avant tout deploiement en production jusqu'aux ameliorations a moyen et long terme.

5.1 Priorite P0 - Correctifs critiques (avant production)

P0.1 - Remplacer les aggregations en memoire par des requetes base de donnees

Les endpoints `/api/dashboard`, `/api/map` et `/api/qos` chargent l'integralite de la table `MesureQoS` en memoire puis effectuent des filtrages et aggregations en JavaScript. Cette approche provoque des problemes de memoire (OOM) et des temps de reponse inacceptables des que le volume de donnees depasse quelques milliers de mesures. La solution consiste a remplacer ces operations par des requetes Prisma `groupBy()` ou du SQL brut qui deleguent l'aggregation au moteur de base de donnees.

Implementation concrete :

1. Pour `/api/dashboard` : Remplacer `findMany()` + `reduce()` par `groupBy({ by: ['regionId', 'operateurId'], _avg, _count })` avec des filtres Prisma `where`.
2. Pour `/api/map` : Utiliser une requete `groupBy` par region avec `_avg` et `_count` au lieu du filtrage JS.
3. Pour `/api/qos` : Consolider les 4 requetes paralleles en 1-2 requetes optimisees avec des sous-requetes.
4. Pour le bulk import (`PUT /api/mesures`) : Pre-charger les operateurs et regions en Map, resoudre les IDs par lookup au lieu de requetes `N+1`, puis utiliser `createMany()` pour l'insertion.

P0.2 - Ajouter les index de base de donnees

Ajouter les index Prisma manquants sur les colonnes les plus filterees. Ces index reduiront les temps de requete de $O(n)$ a $O(\log n)$ pour les recherches et les jointures. L'impact est immediat et mesurable sur les performances de tous les endpoints de lecture.

Modele	Index a ajouter	Justification
MesureQoS	<code>@@index([operateurId])</code>	Filtre principal dans tous les dashboards
MesureQoS	<code>@@index([regionId])</code>	Filtre par region dans map et QoS
MesureQoS	<code>@@index([campagneId])</code>	Jointure campagne-mesures
MesureQoS	<code>@@index([timestamp])</code>	Filtre temporel dans tous les endpoints

MesureQoS	@@index([operateurId, regionId])	Requete composite frequente
AuditLog	@@index([userId]), @@index([createdAt])	Filtre par utilisateur et date
Alerte	@@index([operateurId]), @@index([createdAt])	Filtre par operateur et date
DataAccessPolicy	@@unique([roleType, resource])	Prevenir les politiques en doublon

Tableau 8 : Index de base de donnees recommandes

P0.3 - Implementer le rate limiting sur tous les endpoints

Etendre le rate limiting au-dela du seul endpoint de login. Les endpoints publics (POST /api/alerts avec SIGNALEMENT_PUBLIC) et les endpoints de modification (POST/PUT/PATCH) doivent etre proteges. Utiliser une solution persistante (Redis avec ioredis, ou Upstash Ratelimit pour les deploiements serverless) au lieu du Map en memoire. Ajouter un CAPTCHA ou une verification sur le formulaire public de signalement. Configurer des limites differentes par type d'operation : lecture (100/min), ecriture (30/min), import en masse (5/min), creation de signalement public (3/min).

P0.4 - Ajouter les en-tetes de securite manquants

Ajouter les en-tetes de securite manquants dans le middleware : Content-Security-Policy (CSP) avec une politique restrictive (default-src 'self', script-src 'self', style-src 'self' 'unsafe-inline', img-src 'self' data: https://unpkg.com), Strict-Transport-Security (max-age=31536000; includeSubDomains; preload) pour les deploiements HTTPS, et Cross-Origin-Opener-Policy (same-origin). Remplacer la fonction stripHtml() par une bibliotheque d'assainissement propre comme sanitize-html ou DOMPurify pour une protection XSS adequate.

P0.5 - Ajouter les Error Boundaries React

Ajouter un fichier error.tsx a la racine du repertoire app/ pour capturer les erreurs de rendu React. Sans Error Boundary, toute erreur de rendu dans un composant dashboard provoque un ecran blanc sans possibilite de recuperation pour l'utilisateur. Ajouter egalement un fichier not-found.tsx pour les routes inexistantes. Envelopper chaque dashboard dans une Error Boundary individuelle pour isoler les pannes et permettre aux autres onglets de continuer a fonctionner.

5.2 Priorite P1 - Correctifs importants (semaine 1-2)

P1.1 - Verrouillage de compte et protection timing

Implementer un mecanisme de verrouillage de compte apres 5 tentatives echouees consecutive, avec un deblocage automatique apres 30 minutes. Ajouter un champ failedLoginAttempts et lockedUntil au

modele User dans le schema Prisma. Pour corriger la vulnerabilite de timing, utiliser `bcrypt.compare()` meme quand l'utilisateur n'existe pas : si `findUnique` renvoie null, comparer un hash fictif pour maintenir un temps de reponse constant d'environ 100ms.

P1.2 - Appliquer le RLS sur les operations d'ecriture

Verifier que l'utilisateur peut ecrire pour l'operateur et la region specifiques dans chaque operation POST/PUT/PATCH. Actuellement, les permissions sont verifiees (campaign:write, etc.) mais pas la portee des donnees. Un `INGENIEUR_RF` avec un scope `own_region` peut creer des mesures pour n'importe quelle region. Ajouter une fonction `checkWriteScope(user, resource, targetOperatorId, targetRegionId)` qui verifie que les IDs cibles sont dans les listes d'accès de l'utilisateur.

P1.3 - Implementer le logging structure

Remplacer `console.error` par un logger structure (Pino ou Winston) avec les champs suivants pour chaque entree : `timestamp`, `level`, `requestId` (ID de correlation), `userId`, `method`, `path`, `duration`, `errorCode`, `message`. Ajouter un middleware qui genere un `requestId` unique pour chaque requete et le propage dans tous les logs et les reponses (en-tete `X-Request-Id`). Integrer un APM comme Sentry pour capturer les exceptions non gerees, les erreurs de requete, et les transactions de performance.

P1.4 - Configurer la pipeline CI/CD

Creer un workflow GitHub Actions avec les etapes suivantes : lint (eslint), type-check (`tsc --noEmit`), tests unitaires (Vitest), tests d'integration (API + DB), build Docker, et deploiement. Commencer par les tests les plus critiques : validation RBAC/RLS (chaque role peut-il acceder uniquement a ses donnees ?), validation des schemas Zod, et verification des codes de statut HTTP. Ajouter progressivement des tests E2E avec Playwright pour les flux critiques (login, creation de campagne, import de mesures, generation de rapport).

P1.5 - Corriger la securite de type TypeScript

Remplacer les 44 instances de `'as Record'` par des acces type-safe via les types definis dans `next-auth.d.ts`. Creer un repertoire `src/types/` avec des interfaces partagees pour les reponses API (`DashboardResponse`, `QoSResponse`, `MapResponse`, etc.), les modeles de donnees (`Operator`, `Region`, `Measurement`), et les parametres de requete. Utiliser Zod pour valider les reponses API cote client et non seulement cote serveur.

5.3 Priorite P2 - Ameliorations a moyen terme (mois 1-2)

P2.1 - Migration vers PostgreSQL

SQLite est adequat pour le developpement mais inadapte a la production pour une plateforme nationale. Ses limitations incluent : ecriture single-writer (serialise les ecritures concurrentes), pas de support pour les requetes paralleles, pas de replication, et pas de haute disponibilite. Migrer vers PostgreSQL avec le meme schema Prisma (la migration est transparente avec Prisma Migrate). Cela permettra le scaling horizontal, les requetes paralleles, la replication, et les index avances (GIN, partial indexes). Combiner avec PgBouncer pour le pool de connexions.

P2.2 - Implementer React Query et le code-splitting

Utiliser React Query (deja installe) pour la gestion des donnees serveur avec cache, revalidation en arriere-plan, et optimistic updates. Configurer des staleTime differents par type de donnee : dashboard KPIs (30s), QoS metrics (60s), scores (5min), regions (30min). Implementer React.lazy + Suspense pour chaque dashboard afin de reduire le bundle initial. Supprimer les dependances inutilisees (zustand, next-intl, @mdxeditor/editor, @dnd-kit, react-markdown, react-syntax-highlighter) pour reduire le bundle de 200-400 Ko.

P2.3 - Implementer les formulaires avec react-hook-form + Zod

Migrer tous les formulaires (login, signalement public, creation d'utilisateur, creation de campagne) vers react-hook-form avec @hookform/resolvers/zod pour une validation declaree et type-safe. Ajouter un indicateur de force de mot de passe, des messages d'erreur contextuels, et une validation de format (email, telephone). Utiliser les composants Form de shadcn/ui (deja installes) pour une UX coherente.

P2.4 - Ajouter l'accessibilite (a11y)

Ajouter les attributs ARIA aux composants customises : role='navigation' et aria-label sur la barre laterale, role='tablist'/'tab'/'tabpanel' sur le systeme d'onglets, aria-expanded et aria-haspopup sur les dropdowns, aria-label et title sur les graphiques SVG. Ajouter un lien 'allez au contenu principal' en debut de page. Implementer un piege de focus dans la modale de login. Ajouter des alternatives textuelles aux indicateurs de couleur (severite des alertes, statut des operateurs). Tester avec un lecteur d'ecran (NVDA/VoiceOver) et viser la conformite WCAG 2.1 niveau AA au minimum.

P2.5 - Ajouter le cache de reponses API

Implementer un cache de reponses API avec stale-while-revalidate. Les donnees de dashboard, carte et scoring changent peu d'une minute a l'autre et pourraient etre cachees pendant 1-5 minutes. Utiliser les en-tetes Cache-Control (max-age=60, stale-while-revalidate=300) ou un cache en memoire avec Redis. Supprimer les valeurs Math.random() dans /api/reports qui invalident tout cache potentiel. Ajouter des timestamps de derniere mise a jour dans les reponses pour permettre une invalidation selective.

5.4 Priorite P3 - Ameliorations a long terme (mois 3+)

Amelioration	Description	Effort
API versioning	Prefixer les routes avec /api/v1/ pour permettre des evolutions retrocompatibles	Moyen
Internationalisation	Utiliser next-intl (deja installe) pour externaliser les textes et supporter d'autres langues	Eleve
Real-time updates	Integrer le WebSocket existant dans exemples/ pour les mises a jour en temps reel des mesures	Moyen
Documentation API	Generer la documentation OpenAPI/Swagger a partir des schemas Zod existants	Faible
Soft deletes	Ajouter un champ deletedAt aux modeles et filtrer par default au lieu de supprimer physiquement	Faible
Changement de mot de passe	Ajouter un endpoint PATCH /api/users/me/password pour que les utilisateurs changent leur mot de passe	Faible
2FA/MFA admin	Ajouter TOTP comme seconde facteur d'authentification pour les roles admin et DG	Moyen
Routage fichier-systeme	Migrer de la SPA a un routage Next.js par fichiers pour ameliorer le SEO et le partage de liens	Eleve
Data retention	Implementer des politiques de retention automatique (archive apres 2 ans, suppression apres 5 ans)	Moyen

Tableau 9 : Ameliorations a long terme

6. Plan d'action par phases

Le plan d'action suivant organise les recommandations en phases chronologiques avec des objectifs clairs et mesurables pour chaque phase. L'approche est progressive : les correctifs critiques sont appliques en premier pour securiser la plateforme, puis les ameliorations architecturales sont mises en oeuvre pour garantir la scalabilite et la maintenabilite a long terme.

Phase	Duree	Objectifs	Livrables
Phase 0 : Urgences	1 semaine	Corriger les vulnerabilites de securite critiques et les problemes de performance bloquants	Index DB, rate limiting Redis, CSP + HSTS headers, Error Boundaries, aggregations DB
Phase 1 : Stabilisation	2 semaines	Stabiliser la plateforme avec les correctifs de securite et les fondations de qualite	Verrouillage compte, RLS write, logging structure, CI/CD de base, types TypeScript

Phase 2 : Modernisation	4 semaines	Moderniser l'architecture pour la scalabilité et l'expérience développeur	PostgreSQL, React Query, code-splitting, react-hook-form, a11y, cache API
Phase 3 : Maturation	4 semaines	Compléter les fonctionnalités manquantes et préparer la plateforme pour le long terme	API versioning, i18n, temps réel, documentation API, 2FA, data retention

Tableau 10 : Plan d'action par phases